

第3章 关系模型与关系 规范化理论

人工智能学院
古 晶



第3章 关系模型与关系规范化理论

- 3.1 关系模型及其定义
- 3.2 关系代数及其运算
- 3.3 数据库设计的规范化
- 3.4 知识点小结
- 课堂测试



第3章 关系模型与关系规范化理论

- 以**关系模型**为基础的关系数据库是目前应用较为广泛的数据库。
- 关系模型建立在**数学理论**的基础上。
- 关系数据库的**规范化理论**是数据库逻辑结构设计的理论指南。
- 本章首先介绍了关系模型的基本概念、关系的数学定义、**关系模型的三个要素**以及关系代数等内容。然后主要讨论关系数据库规范化理论，讨论如何设计好的关系模式。



第3章 关系模型与关系规范化理论

- 3.1 关系模型及其定义
- 3.2 关系代数及其运算
- 3.3 数据库设计的规范化
- 3.4 知识点小结
- 课堂测试



3.1 关系模型及其定义

- 关系模型是数据库使用的一种典型数据模型。
- 在关系模型中，其数据结构为具有一定特征的**二维表**。
- 在关系数据库中，数据是以**关系表**形式存储实体数据。
- 关系是一个由行和列组成的二维表。



3.1 关系模型及其定义

□关系

- 单一的数据结构——**关系**

无论是实体集，还是实体集之间的联系均由单一的关系表示。

- 逻辑结构——**二维表**。

从用户的角度来说关系模型的逻辑结构就是一张二维表。

- 建立在**集合代数**的基础上。
- 元数据：关于数据库**结构**的数据。如：表名，列名，表列的属性等。



3.1 关系模型及其定义

□ 关系中基本术语

- **元组**(Tuple): 元组也称记录, 关系表中的每行对应一个元组, 组成元组的元素称为分量。数据库中的一个实体或实体之间的一个联系均使用一个元组来表示。

姓名	性别	专业
史丹妮	女	会计
周冬元	男	商务
李晓辉	女	商务

- **例**: 在上表中有3个元组, 分别对应3个学生, “史丹妮, 女, 会计” 是一个元组, 由 3 个分量组成。



3.1 关系模型及其定义

➤ 属性 (Attribute)

- 关系中的每列对应一个域。由于域可以相同，因此为了加以区分，必须给每列一个命名，这个命名就称为属性。N目关系必有n个属性。
- 属性具有型和值两层含义：**型**指字段名和属性值域；**值**是指属性具体的取值。
- 关系中的字段名具有**标识列**的作用，所以在同一个关系中的字段名(列名)不能相同。**一个关系中通常有多个属性**，属性用于表示实体的特征。



3.1 关系模型及其定义

➤ 候选码 (Candidate key)

- 若关系中的某一属性或属性组的值能**唯一地**标识一个元组，则称该**属性或属性组**为候选码 (Candidate key)。

➤ 主码 (Primary key)

- 若一个关系中有多个候选码，则选定其中一个为**主码**（也可以称为主键，主关键字）。
- 当包含两个或更多个的键称为复合码（键）



3.1 关系模型及其定义

➤ 主码 (Primary key)

- 不仅可以标识唯一的行，还可以建立与别的表之间的联系。
- 作用：
 1. 作为关联表的外键，链接两个表；
 2. 使用主码值来组织关系的存储；
 3. 使用主码索引快速检索数据。



3.1 关系模型及其定义

➤ 主码选择的注意事项：

(1) 建议取值简单的关键字为主码。

(2) 为便于表的维护，不建议使用复合主键。

(3) 代理键

- **定义**：一个没有实际意义的字段作为表的主键。
- **取值**：建议由DBMS（例如MySQL）或者应用程序自动生成，避免人工录入时人为操作产生的错误。



3.1 关系模型及其定义

➤ 全码 (All-key)

- 关系模式的**所有属性**是这个关系模式的候选码，称为全码。

➤ 主属性 (Prime attribute) 和 非主属性 (Non-prime attribute)

- 在关系中，**候选码**中的属性称为主属性，不包含在任何候选码中的属性称为非主属性。



3.1 关系模型及其定义

➤ 代理键

- 是具有DBMS分配的**唯一标识符**，该标识符已经作为**主键**添加到表中。每次创建行时由DBMS分配代理键的唯一值，通常是较短的数字，该值永远不变。
- 该值对于用户没有任何意义。
- **例如**：MySQL数据库使用**AUTO_INCREMENT**函数自动分配代理键的数值。在AUTO_INCREMENT中，起始值可以是任意值（默认为1），但增量总是1。



3.1 关系模型及其定义

□ 数据库中关系的**类型**:

- **基本表**（基本关系或者基表）：实际存在的表，它是实际存储数据的逻辑表示。
- **查询表**：查询结果表或查询中生成的临时表。
- **视图表**：由**基本表**或其他**视图表**导出的表，是**虚表**，不对应实际存储的数据。



3.1 关系模型及其定义

□ 关系的性质:

- (1) 关系中的元组存储了某个实体或实体某个部分的数据。
- (2) 关系中元组的位置具有**顺序无关性**，即元组的顺序可以任意交换。
- (3) 同一属性的数据具有**同质性**，即每一列中的分量是同一类型的数据，它们来自同一个域。
- (4) 同一关系的字段名具有**不可重复性**，即同一关系中不同属性的数据可出自同一个域，但不同的属性要给予不同的字段名。



3.1 关系模型及其定义

□ 关系的性质:

- (5)关系具有元组**无冗余性**，即关系中的任意两个元组不能完全相同。
- (6)关系中**列**的位置具有顺序无关性，即列的次序可以任意交换、重新组织。
- (7)关系中每个分量必须取**原子值**，即每个分量都必须是不可分的数据项。



关系模式



□ 关系模式 (Relation Schema) 是型

□ 关系是值

□ 关系模式是对关系的描述

- 元组集合的结构
- 属性构成
- 属性来自的域
- 属性与域之间的映象关系
- 元组语义以及完整性约束条件
- 属性间的数据依赖关系集合



关系模式形式化定义

□ 形式化定义： $R(U, D, DOM, F)$

□ 各部分的含义：

- R : 关系名
- U : 组成该关系的属性名集合
- D : 属性组 U 中属性所来自的域
- DOM : 属性向域的映象集合
- F : 属性间的数据依赖关系集合



关系模式形式化定义

- **关系模式**通常可以**简记**为 $R(U)$ 或 $R(A_1, A_2, \dots, A_n)$ 。
其中 R 为关系名, $A_1, A_2, \dots, A_n$ 为字段名。
- 域名及属性向域的映像通常直接称为属性的类型及长度。
- 关系和关系模式的**联系与区别**:
 - 关系模式是关系的**框架**或**结构**, 是**静态**的、**稳定**的。
 - 关系是按关系模式组合的**表格**, 关系既包括**结构**也包括其**数据**, 关系的数据是**动态**的、随时间不断变化的。
 - 关系是关系模式在某一时刻的状态或内容。
 - 实际应用中, 人们通常把关系模式和关系都称为关系。



关系模式形式化定义

□ 关系数据库中关于表的三组术语的对应关系：

关系 ↕	元组 ↕	属性 ↕
表 ↕	行 ↕	列 ↕
文件 ↕	记录 ↕	字段 ↕



□ 关系操作的**三大功能**:

- (1)**数据查询**: 数据检索、统计、排序、分组以及用户对信息的需求等功能。
- (2)**数据维护**: 数据添加、删除、修改等数据自身更新的功能。
- (3)**数据控制**: 为了保证数据的安全性和完整性而采用的数据存取控制及并发控制等功能。



关系操作

- 关系操作的**数据查询**和**数据维护**功能使用关系代数中的8种操作来表示, 即并(Union)、差(Difference)、交 (Intersection)、 广义的笛卡儿积 (Extended Cartesian Product)、选择(Select)、投影(Project)、连接(Join)和除(Divide)。
- 其中**选择、投影、并、差、笛卡尔积**是**5种基本操作**。其他操作可以由基本操作导出。



关系操作语言的种类

- 在关系模型中，关系数据库操作通常是用代数方法或逻辑方法实现，分别称为**关系代数**和**关系演算**。
- 关系操作语言可以分为3类：
 - (1)关系代数语言
 - (2)关系演算语言
 - (3)具有关系代数和关系演算双重特点的语言



关系操作语言的种类

- **关系代数语言**：用对**关系的运算**来表达查询要求的语言。
- **关系演算语言**：用查询得到的**元组**应满足的谓词条件来表达查询要求的语言。可以分为**元组关系演算语言**和**域关系演算语言**两种。
- 具有关系代数和关系演算**双重特点**的语言。**结构化查询语言(Structure Query Language, SQL)**是介于关系代数和关系演算之间的语言，它包括**数据定义、数据操作和数据控制** 3 种功能，具有语言简洁、易学易用的特点，是关系数据库的标准语言。



关系完整性



- 关系模型的**完整性规则**是对关系的某种**约束条件**。
- 关系模型允许定义**3类完整性约束**:
 - **实体完整性**
 - **参照完整性**
 - 用户自定义的完整性。——应用领域需要遵循的约束条件，体现了具体领域中的语义约束。

- 关系模型必须满足的完整性约束条件，称为两个不变性。
- 由关系系统自动支持。



实体完整性(Entity Integrity)

- **实体完整性规则**：若属性A是基本关系R的主属性，则属性A**不能取空值**。
 - **例如**，学生关系“学生(学号, 姓名, 性别, 专业号, 年龄)”中，“学号”为主码，则“学号”不能取空值。
- 实体完整性规则规定基本关系的主码不能取空值，**若主码由多个属性组成**，则所有这些属性**都不可以取空值**。
 - **例如**，学生选课关系“选修(学号, 课程号, 成绩)”中，“学号、课程号”为主码，则“学号”和“课程号”两个属性都不能取空值。



实体完整性(Entity Integrity)

□ 对**实体完整性规则**的说明：

- (1) 实体完整性规则是针对**基本关系**而言的。一个基本表通常对应信息世界的一个实体集。
- (2) 信息世界中的**实体**是**可区分的**，即它们具有某种唯一性标识。
- (3) 关系模型中以主码作为**唯一性**标识。
- (4) 主码中的属性即主属性**不能取空值**。



参照完整性(Referential Integrity)

- **参照完整性规则**：若属性(或属性组) F 是基本关系 R 的外码，它与基本关系 S 的主码 K_s 相对应(基本关系 R 和 S 有可能是同一关系)，则对于 R 中每个元组在 F 上的值必须为以下值之一。
 - (1)取**空值**(F 的每个属性值均为空值)。
 - (2)等于 S 中某个元组的**主码值**。
- 主码（主键）与外码（外键）的**列名不一定相同**，唯一的要求是它们的值的**域**必须相同。



参照完整性(Referential Integrity)

□ 实例：

- 学生关系和专业关系表示如下，其中**主码**用**下划**

线标识：

学生(学号， 姓名， 性别， 专业号， 年龄)

专业(专业号， 专业名)

- “专业号” 属性只能取下面两类值：
 - **空值**， 表示尚未给该学生分配专业。
 - **非空值**， 这时该值必须是专业关系中某个元组的“专业号” 值。



用户定义的完整性(User-defined Integrity)

- 针对某一具体关系数据库的约束条件的，它反映某一具体应用所涉及的数据必须满足的语义要求。
- DBMS可以为用户实现如下**自定义完整性约束**：
 - (1) **定义域**的数据类型和取值范围。
 - (2) **定义属性**的数据**类型**和取值**范围**。
 - (3) 定义属性的**缺省值**。
 - (4) 定义属性是否允许空值。
 - (5) 定义属性取值**唯一性**。
 - (6) 定义属性间的数据依赖性。



第3章 关系模型与关系规范化理论

- 3.1 关系模型及其定义
- 3.2 关系代数及其运算
- 3.3 数据库设计的规范化
- 3.4 知识点小结
- 课堂测试



3.2 关系代数及其运算

- 关系代数是一种抽象的**查询语言**。
- 是关系数据操作语言的一种传统表达方式，它是用对关系的运算来表达查询的。
- 关系代数是建立在**集合代数**的基础上。



关系的数学定义

□ 关系数据库结构的**形式化定义**:

- (1) **域** (Domain)
- (2) **笛卡尔积** (Cartesian Product)
- (3) **关系** (Relation)



关系的数学定义

□ 域 (Domain)

- 域是一组具有**相同数据类型**值的集合。
- 在关系模型中，使用域来表示**实体属性的取值范围**。
- 通常用 D_i 表示某个域。
- 例如：
 - 整数
 - 实数
 - 介于某个取值范围的整数
 - 长度指定长度的字符串集合
 - {‘男’ , ‘女’ }
 -



关系的数学定义

□ 笛卡儿积(Cartesian Product)

- 给定一组域 $D_1, D_2, \dots, D_n$, 这些域中可以有相同的。

$D_1, D_2, \dots, D_n$ 的笛卡尔积为:

$$D_1 \times D_2 \times \dots \times D_n =$$

$$\{ (d_1, d_2, \dots, d_n) \mid d_i \in D_i, i = 1, 2, \dots, n \}$$

- 所有域的所有取值的一个组合
- 不能重复



关系的数学定义

➤ 元组 (Tuple)

- 笛卡尔积中**每一个元素** $(d_1, d_2, \dots, d_n)$ 叫作一个n元组 (n-tuple) , 或简称元组(Tuple)。
- 例:** (张清玫, 计算机专业, 李勇)、(张清玫, 计算机专业, 刘晨)等都是元组。

➤ 分量 (Component)

- 笛卡尔积元素 $(d_1, d_2, \dots, d_n)$ 中的**每一个值** d_i 叫作一个分量。
- 例:** 张清玫、计算机专业、李勇、刘晨等都是分量。



关系的数学定义

➤ 基数 (Cardinal number)

- 若 D_i ($i = 1, 2, \dots, n$) 为有限集, 其基数为 m_i ($i = 1, 2, \dots, n$)

- 则 $D_1 \times D_2 \times \dots \times D_n$ 的基数 M 为:
$$M = \prod_{i=1}^n m_i$$

➤ 笛卡尔积的表示方法

- 笛卡尔积可表示为一个**二维表**。
- 表中的**每行**对应一个**元组**, 表中的**每列**对应一个**域**。



笛卡尔积实例

➤ 给出3个**域**:

姓名集合: $D1 = \{\text{史丹妮, 周冬元, 李晓辉}\}$

性别集合: $D2 = \{\text{男, 女}\}$

专业集合: $D3 = \{\text{会计, 商务}\}$

➤ **笛卡尔积**的结果:

$D1 \times D2 \times D3 = \{ (\text{史丹妮, 男, 会计}), (\text{史丹妮, 男, 商务}), (\text{史丹妮, 女, 会计}), (\text{史丹妮, 女, 商务}), (\text{周冬元, 男, 会计}), (\text{周冬元, 男, 商务}), (\text{周冬元, 女, 会计}), (\text{周冬元, 女, 商务}), (\text{李晓辉, 男, 会计}), (\text{李晓辉, 男, 商务}), (\text{李晓辉, 女, 会计}), (\text{李晓辉, 女, 商务}) \}$



笛卡尔积实例

➤ 笛卡尔积结果表:

姓名 ↴	性别 ↴	专业 ↴
史丹妮 ↴	男 ↴	会计 ↴
史丹妮 ↴	男 ↴	商务 ↴
史丹妮 ↴	女 ↴	会计 ↴
史丹妮 ↴	女 ↴	商务 ↴
周冬元 ↴	男 ↴	会计 ↴
周冬元 ↴	男 ↴	商务 ↴
周冬元 ↴	女 ↴	会计 ↴
周冬元 ↴	女 ↴	商务 ↴
李晓辉 ↴	男 ↴	会计 ↴
李晓辉 ↴	男 ↴	商务 ↴
李晓辉 ↴	女 ↴	会计 ↴
李晓辉 ↴	女 ↴	商务 ↴



关系的数学定义

□ 关系

- 关系为**多个域的笛卡尔乘积**的**有限子集**，是一张二维表。
- $D_1 \times D_2 \times \dots \times D_n$ 的子集叫作在域 $D_1, D_2, \dots, D_n$ 上的关系，表示为：

$$R(D_1, D_2, \dots, D_n)$$

R: 关系名

n: 关系的目或度 (Degree)

- 当 $n=1$ 时，称该关系为**单目关系**(Unary relation);
- 当 $n=2$ 时，称该关系为**二目关系**(Binary relation)。



关系实例



- 从上述D1,D2,D3的笛卡儿积中取出一个子集来构造一个学生关系。
- 由于一个学生只有一个专业和性别，所以笛卡儿积中的许多元组在实际中是无意义的，仅仅挑出有实际意义的元组构建一个关系，该关系名为 Student，字段名取域名：姓名，性别和专业。

姓名 ↴	性别 ↴	专业 ↴
史丹妮 ↴	女 ↴	会计 ↴
周冬元 ↴	男 ↴	商务 ↴
李晓辉 ↴	女 ↴	商务 ↴



关系代数概述



- 运算的**三要素**：运算对象、运算符、运算结果。
- 关系代数的**运算对象**是**关系**，**运算结果**亦为**关系**。
- 关系代数中使用的运算符包括4类：**集合运算符**、**专门的关系运算符**、**比较运算符**和**逻辑运算符**。
- 关系代数的运算按**运算符**的不同可分为：
 - 传统的**集合运算**：将关系看成元组的集合，其运算是从关系的“水平”方向即**行**的角度进行的。
 - 专门的**关系运算**：不仅涉及**行**而且涉及**列**。比较运算符和逻辑运算符是用来辅助专门的关系运算进行操作的。



关系代数运算符

运算符		含义	运算符		含义
集合 运算符	$\cup$	并	比 较 运 算 符	$>$	大于
	$-$	差		$\geq$	大于等于
	$\cap$	交		$<$	小于
	$\times$	广义笛卡儿积		$\leq$	小于等于
				$=$	等于
				$\neq$	不等于
专门的 关系运 算符	σ	选择	逻辑 运算 符	$\neg$	非
	π	投影		$\wedge$	与
	$\bowtie$	连接		$\vee$	或
	$\div$	除			



传统的集合运算



- 传统的**集合运算**是**二目运算**，包括**并**、**交**、**差**、**广义笛卡儿积**4种运算。
- 设关系R和关系S具有相同的目n(即两个关系都具有n个属性)，且相应的属性取自同一个域，则可以定义并、差、交、广义笛卡儿积运算如下：

1. **并(Union)**

关系R与关系S的并记作：

$$R \cup S = \{t \mid t \in R \vee t \in S\}, \quad t \text{ 是元组变量}$$

其结果关系仍为 n 目关系，由属于 R **或**属于 S 的元组组成。



传统的集合运算



2. 差(Difference)

关系 R 与关系 S 的差记作:

$$R-S = \{t \mid t \in R \wedge t \notin S\}, \text{ } t \text{ 是元组变量}$$

其结果关系仍为 n 目关系, 由属于 R 而**不**属于 S 的所有元组组成。

3. 交(Intersection)

关系 R 与关系 S 的交记作:

$$R \cap S = \{t \mid t \in R \wedge t \in S\}, \text{ } t \text{ 是元组变量}$$

其结果关系仍为 n 目关系, 由既属于 R **又** 属于 S 的元组组成。关系的交可以用差来表示, 即 $R \cap S = R - (R - S)$



传统的集合运算

4. 广义笛卡儿积(Extended Cartesian Product)

两个分别为n目和m目的关系R和S的广义笛卡儿积是一个(n+m)列的元组的集合。元组的前n列是关系R的一个元组，后m列是关系S的一个元组。若R有k1个元组，S有k2个元组，则关系R和关系S的广义笛卡儿积有 $k1 \times k2$ 个元组。记作：

$$R \times S = \{tr \frown ts \mid Tr \in R \wedge Ts \in S\}$$



传统的集合运算（举例）

□ R和S

- 具有相同的目 n （即两个关系都有 n 个属性）
- 相应的属性取自同一个域

R		
A	B	C
a_1	b_1	c_1
a_1	b_2	c_2
a_2	b_2	c_1

S		
A	B	C
a_1	b_2	c_2
a_1	b_3	c_2
a_2	b_2	c_1

$R \cup S$		
A	B	C
a_1	b_1	c_1
a_1	b_2	c_2
a_2	b_2	c_1
a_1	b_3	c_2

$R - S$		
A	B	C
a_1	b_1	c_1



传统的集合运算（举例）

R		
A	B	C
a_1	b_1	c_1
a_1	b_2	c_2
a_2	b_2	c_1

S		
A	B	C
a_1	b_2	c_2
a_1	b_3	c_2
a_2	b_2	c_1

$R \cap S$		
A	B	C
a_1	b_2	c_2
a_2	b_2	c_1

$R \times S$					
$R.A$	$R.B$	$R.C$	$S.A$	$S.B$	$S.C$
a_1	b_1	c_1	a_1	b_2	c_2
a_1	b_1	c_1	a_1	b_3	c_2
a_1	b_1	c_1	a_2	b_2	c_1
a_1	b_2	c_2	a_1	b_2	c_2
a_1	b_2	c_2	a_1	b_3	c_2
a_1	b_2	c_2	a_2	b_2	c_1
a_2	b_2	c_1	a_1	b_2	c_2
a_2	b_2	c_1	a_1	b_3	c_2
a_2	b_2	c_1	a_2	b_2	c_1



专门的关系运算

□ 专门的**关系运算**包括**选择**、**投影**、**连接**、**除**等。

□ 为了叙述上的方便，先引入几个**记号**：

(1) 设关系模式为 $R(A_1, A_2, \dots, A_n)$ ，它的一个关系设为 R ， $t \in R$ 表示 t 是 R 的一个元组， $t[A_i]$ 表示元组 t 中相应于属性 A_i 上的一个分量。

(2) 若 $A = \{A_{i1}, A_{i2}, \dots, A_{ik}\}$ ，其中 $A_{i1}, A_{i2}, \dots, A_{ik}$ 是 $A_1, A_2, \dots, A_n$ 中的一部分，则 A 称为**字段名**或**域列**。 $t[A] = (t[A_{i1}], t[A_{i2}], \dots, t[A_{ik}])$ 表示元组 t 在字段名 A 上诸分量的集合。 A 表示 $\{A_1, A_2, \dots, A_n\}$ 中去掉 $\{A_{i1}, A_{i2}, \dots, A_{ik}\}$ 后剩余的属性组。



专门的关系运算

(3) R 为 n 目关系, S 为 m 目关系。 $tr \in R$, $ts \in S$, $tr \frown ts$ 称为**元组的连接**, 它是一个 $n+m$ 列的元组, 前 n 个分量为 R 中的一个 n 元组, 后 m 个分量为 S 中的一个 m 元组。

(4) 给定一个关系 $R(X, Z)$, X 和 Z 为属性组。定义当 $t[X]=x$ 时, x 在 R 中的**象集**为:

$$Z_x = \{t[Z] \mid t \in R, t[X]=x\}$$

它表示 R 中属性组 X 上值为 x 的诸元组在 Z 上分量的集合。



专门的关系运算



□ 象集举例：

- x_1 在 R 中的象集

$$Z_{x_1} = \{Z_1, Z_2, Z_3\}$$

- x_2 在 R 中的象集

$$Z_{x_2} = \{Z_2, Z_3\}$$

- x_3 在 R 中的象集

$$Z_{x_3} = \{Z_1, Z_3\}$$

R

x_1	Z_1
x_1	Z_2
x_1	Z_3
x_2	Z_2
x_2	Z_3
x_3	Z_1
x_3	Z_3



专门的关系运算

□ 选择(Selection)

➤ 选择又称为**限制** (Restriction)

➤ 选择运算符的含义

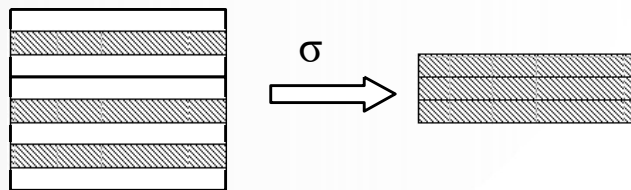
- 在关系R中选择满足**给定条件**的诸元组

$$\sigma_F(R) = \{t \mid t \in R \wedge F(t) = \text{'真'}\}$$

- F表示选择条件，它是一个逻辑表达式，取逻辑值“**真**”或“**假**”

- F的基本形式为： $X_1 \theta Y_1 [\Phi X_2 \theta Y_2 \dots]$

➤ 选择运算是从关系R中选取使逻辑表达式F为真的元组，是从**行**的角度进行的运算：





选择运算举例

- 设有一个**学生-课程数据库**见下表，它包括以下内容：
 - **学生关系** Student(说明：Sno 表示学号，Sname 表示姓名，Ssex 表示性别，Sage 表示年龄，Sdept 表示所在系)
 - **课程关系** Course(说明：Cno 表示课程号，Cname 表示课程名)
 - **选修关系** Score(说明：Sno 表示学号，Cno 表示课程号，Degree 表示成绩)
- 其**关系模式**如下：
 - Student(Sno, Sname, Ssex, Sage, Sdept)
 - Course(Cno, Cname)
 - Score(Sno, Cno, Degree)



选择运算举例

□ Student表

Student 表				
Sno	Sname	Ssex	Sage	Sdept
000101	李晨	男	18	信息系
000102	王博	女	19	数学系
010101	刘思思	女	18	信息系
010102	王国美	女	20	物理系
020101	范伟	男	19	数学系



选择运算举例

□ Course表

Course 表	
Cno	Cname
C1	数学
C2	英语
C3	计算机
C4	制图



选择运算举例

□ Score表

Score 表		
Sno	Cno	Degree
000101	C1	90
000101	C2	87
000101	C3	72
010101	C1	85
010101	C2	42
020101	C3	70



选择运算举例

□ 例1：查询数学系学生的信息。

➤ $\sigma_{Sdept='数学系'}(Student)$ 或 $\sigma_5='数学系'(Student)$

➤ 查询结果：

Sno ↴	Sname ↴	Ssex ↴	Sage ↴	Sdept ↴
000102 ↴	王博 ↴	女 ↴	19 ↴	数学系 ↴
020101 ↴	范伟 ↴	男 ↴	19 ↴	数学系 ↴



选择运算举例

□ 例2：查询年龄小于 20 岁的学生的信息。

➤ $\sigma_{\text{Sage} < 20}(\text{Student})$ 或 $\sigma_{4 < 20}(\text{Student})$

➤ 查询结果：

Sno ↵	Sname ↵	Ssex ↵	Sage ↵	Sdept ↵
000101 ↵	李晨 ↵	男 ↵	18 ↵	信息系 ↵
000102 ↵	王博 ↵	女 ↵	19 ↵	数学系 ↵
010101 ↵	刘思思 ↵	女 ↵	18 ↵	信息系 ↵
020101 ↵	范伟 ↵	男 ↵	19 ↵	数学系 ↵



专门的关系运算

□ 投影 (Projection)

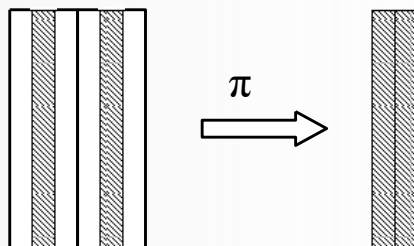
➤ 投影运算符的**含义**

- 从R中选择出若干属性列组成新的关系

$$\pi_A(R) = \{ t[A] \mid t \in R \}$$

A: R中的属性列

- 投影操作主要是从**列**的角度进行运算
- 投影之后不仅取消了原关系中的某些列，而且还可能取消某些元组 (**避免重复行**)





选择运算举例

□ 例1：查询学生的学号和姓名。

➤ $\pi_{\text{Sno}, \text{Sname}}(\text{Student})$ 或 $\pi_{1, 2}(\text{Student})$

➤ 查询结果：

Sno ↵	Sname ↵
000101 ↵	李晨 ↵
000102 ↵	王博 ↵
010101 ↵	刘思思 ↵
010102 ↵	王国美 ↵
020101 ↵	范伟 ↵



专门的关系运算

□ 连接 (Join)

➤ 连接也称为θ连接

➤ 连接运算的含义：

$$R \bowtie_{A\theta B} S = \{ \langle t_r, t_s \rangle \mid t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B] \}.$$

- 从两个关系的笛卡尔积中选取**属性间满足一定条件**的元组。

- A和B：分别为R和S上度数相等且可比的属性组。

- θ：比较运算符。

➤ 从R和S的笛卡尔积 $R \times S$ 中选取R关系在A属性组上的值与S关系在B属性组上值满足比较关系的θ元组。



专门的关系运算

□ 两类最常用也最重要的连接：

➤ **等值连接**

- θ 为 “=” 的连接运算称为等值连接
- 从关系R与S的广义笛卡尔积中选取A、B**属性值相等**的那些元组。

$$R \bowtie_{A=B} S = \{t_r \hat{t}_s \mid t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B]\}$$

➤ **自然连接**

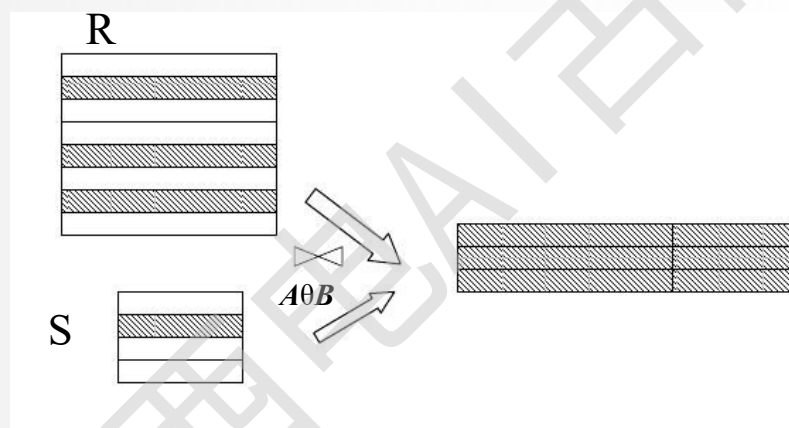
- 一种**特殊的等值连接**
- 要求两个关系中进行比较的分量必须是相同的属性组，并且在结果中把**重复的字段名去掉**
- 若R和S具有相同的属性组B，则记作：

$$R \bowtie S = \{t_r \hat{t}_s \mid t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B]\}$$



专门的关系运算

- 一般连接与自然连接的**区别**:
 - 一般的连接操作是从行的角度进行运算。



- 自然连接还需要**取消重复列**，所以是同时从**行和列**的角度进行运算。



专门的关系运算

➤ 外连接:

- 如果把**舍弃的元组**也保存在结果关系中, 而在其他属性上填空值(Null), 这种连接就叫做外连接 (OUTER JOIN)。

➤ 左外连接:

- 如果只把**左边关系R**中要舍弃的元组保留就叫做左外连接(LEFT OUTER JOIN或LEFT JOIN)

➤ 右外连接:

- 如果只把**右边关系S**中要舍弃的元组保留就叫做右外连接(RIGHT OUTER JOIN或RIGHT JOIN)。



专门的关系运算

➤ 例：关系R和关系S 如下所示：

<i>R</i>		
<i>A</i>	<i>B</i>	<i>C</i>
a_1	b_1	5
a_1	b_2	6
a_2	b_3	8
a_2	b_4	12

<i>S</i>	
<i>B</i>	<i>E</i>
b_1	3
b_2	7
b_3	10
b_3	2
b_5	2

$R \times S$:

<i>A</i>	<i>R.B</i>	<i>C</i>	<i>S.B</i>	<i>E</i>
a_1	b_1	5	b_1	3
a_1	b_1	5	b_2	7
a_1	b_1	5	b_3	10
a_1	b_1	5	b_3	2
a_1	b_1	5	b_5	2
a_1	b_2	6	b_1	3
a_1	b_2	6	b_2	7
a_1	b_2	6	b_3	10
a_1	b_2	6	b_3	2
a_1	b_2	6	b_5	2

.....



专门的关系运算

➤ 一般连接:

一般连接 $R \bowtie_{C < E} S$ 的结果如下:

$R \bowtie_{C < E} S$				
A	$R.B$	C	$S.B$	E
a_1	b_1	5	b_2	7
a_1	b_1	5	b_3	10
a_1	b_2	6	b_2	7
a_1	b_2	6	b_3	10
a_2	b_3	8	b_3	10



专门的关系运算

➤ 等值连接：

等值连接 $R \bowtie S$ 的结果如下：
 $\underline{R.B=S.B}$

A	$R.B$	C	$S.B$	E
a_1	b_1	5	b_1	3
a_1	b_2	6	b_2	7
a_2	b_3	8	b_3	10
a_2	b_3	8	b_3	2



专门的关系运算

➤ 自然连接：

自然连接 $R \bowtie S$ 的结果如下：

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2



专门的关系运算

➤ 外连接：

下图是关系R和关系S的外连接：

<i>A</i>	<i>B</i>	<i>C</i>	<i>E</i>
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL
NULL	b_5	NULL	2

(a) 外连接



专门的关系运算

➤ 左外连接和右外连接：

图(b)是关系R和关系S的左外连接,图(c)是右外连接

<i>A</i>	<i>B</i>	<i>C</i>	<i>E</i>
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL

(b) 左外连接

<i>A</i>	<i>B</i>	<i>C</i>	<i>E</i>
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
NULL	b_5	NULL	2

(c) 右外连接



专门的关系运算

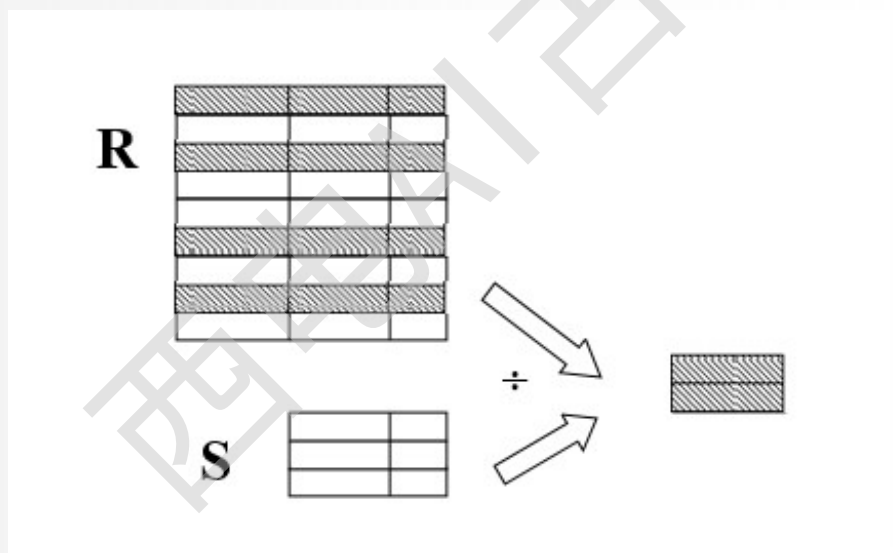
□ 除 (Division)

- 给定关系 $R(X, Y)$ 和 $S(Y, Z)$, 其中 X, Y, Z 为属性组。R中的Y与S中的Y可以有不同的字段名, 但必须出自相同的域集。R与S的除运算得到一个新的关系 $P(X)$, P是R中满足下列条件的元组在X字段名上的投影: 元组在X上分量值 x 的象集 Y_x 包含S在Y上投影的集合。
$$R \div S = \{t_r[X] \mid t_r \in R \wedge \pi_Y(S) \subseteq Y_x\}$$
- 其中 Y_x 为 x 在R中的象集, $x=t_r[X]$ 。



专门的关系运算

- **除**操作是同时从**行和列**角度进行运算的。





专门的关系运算



➤ 关系除法运算分下面4步进行:

- 1) 将被除关系的属性分为**象集属性**和**结果属性**: 与除关系相同的属性属于象集属性, 不相同的属性属于结果属性。
- 2) 在除关系中, 对与被除关系相同的属性 (象集属性) 进行投影, 得到除目标数据集。
- 3) 将被除关系分组, 原则是, **结果属性值**一样的元组分为一组。
- 4) 逐一考察每个组, 如果它的象集属性值中包括除目标数据集, 则对应的结果属性值应属于该除法运算结果集。



专门的关系运算

➤ 例： 设关系 R 、 S 分别为下图的(a)和(b)， $R \div S$ 的结果为图(c)

R		
A	B	C
a_1	b_1	c_2
a_2	b_3	c_7
a_3	b_4	c_6
a_1	b_2	c_3
a_4	b_6	c_6
a_2	b_2	c_3
a_1	b_2	c_1

(a)

S		
B	C	D
b_1	c_2	d_1
b_2	c_1	d_1
b_2	c_3	d_2

(b)

$R \div S$	
A	
a_1	

(c)



专门的关系运算

- 在关系R中, A可以取四个值{a1, a2, a3, a4}
a1的象集为 {(b1, c2), (b2, c3), (b2, c1)}
a2的象集为 {(b3, c7), (b2, c3)}
a3的象集为 {(b4, c6)}
a4的象集为 {(b6, c6)}
- S在(B, C)上的投影为
{(b1, c2), (b2, c1), (b2, c3)}
- 只有a1的象集包含了S在(B, C)属性组上的投影
所以: $R \div S = \{a1\}$



专门的关系运算

➤ 综合实例：

设学生-课程数据库中有 3 个关系：S、C和 SC，利用关系代数进行查询。

学生关系：S(Sno, Sname, SSex, Sage)

课程关系：C(Cno, Cname, Teacher)

学习关系：SC(Sno, Cno, Degree)



举例



(1) 查询学习课程号为 C3 号课程的学生学号和成绩。

$$\pi_{Sno, Degree}(\sigma_{Cno='C3'}(SC))$$

(2) 查询学习课程号为 C4 课程的学生学号和姓名。

$$\pi_{Sno, Sname}(\sigma_{Cno='C4'}(S \bowtie SC))$$

(3) 查询学习课程名为 maths 的学生学号和姓名。

$$\pi_{Sno, Sname}(\sigma_{Cname='maths'}(S \bowtie SC \bowtie C))$$

(4) 查询学习课程号为 C1 或 C3 课程的学生学号。

$$\pi_{Sno}(\sigma_{Cno='C1' \vee Cno='C3'}(SC))$$

(5) 查询不学习课程号为 C2 的学生的姓名和年龄。

$$\pi_{Sname, Sage}(S) - \pi_{Sname, Sage}(\sigma_{Cno='C2'}(S \bowtie SC))$$



第3章 关系模型与关系规范化理论

- 3.1 关系模型及其定义
- 3.2 关系代数及其运算
- 3.3 数据库设计的规范化**
- 3.4 知识点小结
- 课堂测试



3.3 数据库设计的规范化

- 关系数据库的设计主要是**关系模式的设计**，关系模式设计的好坏将直接影响到数据库设计的成败。
- 设计数据库要考虑减少**冗余数据**和避免**数据经常发生变化**，减少额外的维护。
- 规范化的**核心思想**就是表中每个**决定因子**都必须是**候选键**。若不满足，可以将表分解成两个或多个满足条件的表。



问题的提出

□回顾关系模型的**形式化定义**:

关系模式的**完整表示**是一个**五元组**: $R(U, D, Dom, F)$

其中:

R: 关系名, 代表一个关系模式;

U: 关系模式 R 的属性集合(属性组);

D: 属性集合 U 的数据域;

Dom: 属性到域的映射关系;

F: 属性集合 U 上的一组数据依赖的集合。

□ **三元组**: $R(U, F)$, **数据依赖**是关系模式的重要因素。



问题的提出

□ 用一个实例说明如果一个关系没有经过规范化可能会出现的问题：

➤ **例如**，要设计一个教学管理数据库，希望从该数据库中得到学生学号、姓名、年龄、性别、系别、系主任姓名、学生学习的课程名和该课程的成绩信息。

➤ 若将此信息要求设计为一个关系，则**关系模式**为：

$S(sno, sname, sage, ssex, sdept, mname, cno, cname, score)$



问题的提出

□ 该关系模式中各属性之间的**关系**为：

- 一个系有若干学生， 但一个学生只属于一个系；
- 一个系只能有一名系主任， 但一个系主任可以同时兼几个系的系主任；
- 一个学生可以选修多门课程， 每门课程可被若干学生选修；
- 每个学生学习的每门课程都有一个成绩。

□ 分析得到， 此关系模式的**码**为(sno, cno)。



关系模式 S 的实例

sno ↵	sname ↵	sage ↵	ssex ↵	sdept ↵	mname ↵	Cno ↵	cname ↵	score ↵
1414855328 ↵	王芮 ↵	20 ↵	女 ↵	计算机系 ↵	王晨 ↵	C01 ↵	C 语言程序设计 ↵	78 ↵
1414855328 ↵	王芮 ↵	20 ↵	女 ↵	计算机系 ↵	王晨 ↵	C02 ↵	数据结构 ↵	84 ↵
1414855328 ↵	王芮 ↵	20 ↵	女 ↵	计算机系 ↵	王晨 ↵	C03 ↵	数据库原理及应用 ↵	68 ↵
1414855328 ↵	王芮 ↵	20 ↵	女 ↵	计算机系 ↵	王晨 ↵	C04 ↵	数字电路 ↵	90 ↵
2014010225 ↵	王梦瑶 ↵	19 ↵	女 ↵	计算机系 ↵	王晨 ↵	C01 ↵	C 语言程序设计 ↵	92 ↵
2014010225 ↵	王梦瑶 ↵	19 ↵	女 ↵	计算机系 ↵	王晨 ↵	C02 ↵	数据结构 ↵	77 ↵
2014010225 ↵	王梦瑶 ↵	19 ↵	女 ↵	计算机系 ↵	王晨 ↵	C03 ↵	数据库原理及应用 ↵	83 ↵
2014010225 ↵	王梦瑶 ↵	19 ↵	女 ↵	计算机系 ↵	王晨 ↵	C04 ↵	数字电路 ↵	79 ↵
2014010302 ↵	周冬元 ↵	18 ↵	男 ↵	电子系 ↵	秦玉超 ↵	C05 ↵	高等数学 ↵	80 ↵
2014010302 ↵	周冬元 ↵	18 ↵	男 ↵	电子系 ↵	秦玉超 ↵	C06 ↵	机械制图 ↵	83 ↵
2014010302 ↵	周冬元 ↵	18 ↵	男 ↵	电子系 ↵	秦玉超 ↵	C07 ↵	自动控制 ↵	73 ↵
2014010302 ↵	周冬元 ↵	18 ↵	男 ↵	电子系 ↵	秦玉超 ↵	C08 ↵	电工基础 ↵	92 ↵



关系模式 S 存在的问题

1) **数据冗余**太大

每个系名和系主任的名字存储的次数等于该系学生人数乘以每个学生选修的课程门数，系名和系主任数据重复量太大。

2) **插入异常**

一个新系没有招生时，或系里有学生但没有选修课程，系名和系主任名无法插入到数据库中。因为在这个关系模式中码是(sno, cno)，这时没有学生而使得学号无值，或学生没有选课而使得课程名无值。但在一个关系中，码属性不能为空值，因此关系数据库无法操作，导致插入异常。



关系模式 S 存在的问题

3) 删除异常

当某系的学生全部毕业而又没有招新生时，删除学生信息的同时，系及系主任名的信息随之删除，但这个系依然存在，而在数据库中却无法找到该系的信息，即出现了删除异常。

4) 更新异常

若某系换系主任，数据库中该系的学生记录应全部修改。如果稍有不慎，某些记录漏改了，则造成数据的不一致，即出现了更新异常。



关系模式 S 存在的问题

□ 发生插入异常和删除异常的原因

- 该关系模式中属性与属性之间存在**不好的数据依赖**。一个“好”的关系模式应当不会发生插入和删除异常，冗余度要尽可能地少。

□ 检测和消除数据冗余的方法

- 数据库**规范化**
- 规范化是通过**最小化数据冗余**来提升数据库质量的过程，它是基于函数依赖以及一系列范式定义的，最为常用的是第一范式(1NF)、第二范式(2NF)和第三范式(3NF)。



关系模式 S 存在的问题

□ 对于存在问题的关系模式，可以通过**模式分解**的方法使之规范化。

□ 例如将上述关系模式分解成 4 个关系模式：

S(sno, sname, sage, ssex, sdept)

Course (cno, cname)

SC(sno, cno, score)

DEPT(sdept, mname)

这样分解后，4个关系模式都不会发生插入异常、删除异常的问题，数据的冗余也得到控制，数据的更新也变得简单。

□ “**分解**”是解决冗余的主要方法，也是规范化的一条原则，“关系模式有冗余问题，就分解它”。



□ 规范化

- 规范化是指用形式更为简洁、结构更加规范的关系模式取代原有关系模式的过程。关系模式必须满足一定的完整性约束条件以达到现实世界对数据的要求。
- **完整性约束条件**主要包括以下两个方面：
 - (1)对**属性取值范围**的限定。
 - (2)属性值间的**相互联系**(主要体现在值的相等与否),这种联系称为**数据依赖**。



函数依赖



□ 函数依赖

- 是指通过一个关系中属性间值的相等与否体现出来的数据间的相互关系;
- 是现实世界属性间相互联系的抽象;
- 是数据内在的性质。

□ 分类:

- **函数依赖(Functional Dependency, FD)**
- 多值依赖(MultiValued Dependency, MVD)
- 连接依赖(Join Dependency, JD)



函数依赖



【定义 3.1】 设有关系模式 $R(A_1, A_2, \dots, A_n)$ 或简记为 $R(U)$, X, Y 是 U 的子集, r 是 R 的任一具体关系, 如果对 r 的任意两个元组 t_1, t_2 , 由 $t_1[X] = t_2[X]$ 导致 $t_1[Y] = t_2[Y]$, 则称 **X 函数决定 Y** , 或 **Y 函数依赖于 X** , 记为 $X \rightarrow Y$ 。 **$X \rightarrow Y$ 为模式 R 的一个函数依赖。**

这里的 $t_1[X]$ 表示元组 t_1 在属性集 X 上的值, $t_2[X]$ 表示元组 t_2 在属性集 X 上的值, FD 是对关系 R 的一切可能的当前值 r 定义的, 不是针对某个特定关系。通俗地说, 在当前值 r 的两个不同元组中, 如果 X 值相同, 就一定要求 Y 值也相同。或者说, 对于 X 的每一个具体值, 都有 Y 唯一的值与之对应, 即 Y 值由 X 值决定, 因而这种数据依赖称为 **函数依赖**。



函数依赖说明

1. 函数依赖不是指关系模式R的某个或**某些关系实例**满足的约束条件，而是指R的**所有关系实例**均要满足的约束条件。

2. 函数依赖是**语义范畴**的概念。只能根据数据的语义来确定函数依赖。

例如“姓名→年龄”这个函数依赖只有在不允许有同名人的条件下成立

3. **数据库设计者可以对现实世界作强制的规定。**

例如规定不允许同名人的出现，函数依赖“姓名→年龄”成立。所插入的元组必须满足规定的函数依赖，若发现有同名人的存在，则拒绝装入该元组。



相关术语和记号

- (1) $X \rightarrow Y$, 但 $Y \not\subseteq X$, 则称 $X \rightarrow Y$ **是非平凡的函数依赖**。
- (2) $X \rightarrow Y$, 但 $Y \subseteq X$, 则称 $X \rightarrow Y$ 是平凡的函数依赖。因为平凡的函数依赖总是成立的, 所以若不特别声明, 后面提到的函数依赖, 都不包含平凡的函数依赖。
- (3) 若 $X \rightarrow Y$, $Y \rightarrow X$, 则称 $X \leftrightarrow Y$ 。
- (4) 若 Y 不函数依赖于 X , 则记作 $X \nrightarrow Y$ 。



完全函数依赖和部分函数依赖

【定义 3.2】 在关系模式 $R(U)$ 中, 如果 $X \rightarrow Y$, 并且对于 X 的任何一个真子集 X' , 都有 $X' \nrightarrow Y$, 则称 Y 对 X **完全函数依赖**, 记作

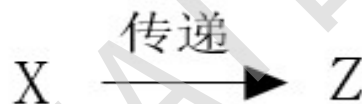
$$X \xrightarrow{F} Y$$

若 $X \rightarrow Y$, 如果存在 X 的某一真子集 X' ($X' \subseteq X$), 使 $X' \rightarrow Y$, 则称 Y 对 X **部分函数依赖**, 记作: $X \xrightarrow{P} Y$



传递函数依赖和直接函数依赖

【定义 3.3】 在关系模式 $R(U)$ 中, X 、 Y 、 Z 是 R 的 3 个不同的属性或属性组, 如果 $X \rightarrow Y$ ($Y \not\subseteq X$, Y 不是 X 的子集), 且 $Y \not\rightarrow X$, $Y \rightarrow Z$, $Z \not\subseteq Y$. 则称 Z 对 X **传递函数依赖**, 记作:



- 传递依赖: 假设 ABC 分别是同一个数据结构 R 中的三个元素或分别是 R 中若干数据元素的集合, 如果 C 依赖 B , 而 B 依赖于 A , 那么 C 自然依赖于 A , 即称 **C 传递依赖 A** 。
- 加上条件 $Y \not\rightarrow X$, 是因为如果 $Y \rightarrow X$, 则 $X \leftrightarrow Y$, 实际上是 $X \rightarrow Z$, 是**直接函数依赖**而不是传递函数依赖。



【定义 3.4】 设 K 是关系模式 $R(U, F)$ 中的属性或属性集合, K' 是 K 的任一真子集。

若 $K \rightarrow U$, 而不存在 $K' \rightarrow U$, 则 K 为 R 的候选码 (Candidate Key), 简称**码**。

(1) 若候选码多于一个, 则选取其中的一个为**主码** (Primary Key)。

(2) 包含在任一候选码中的属性, 称为**主属性** (Prime Attribute) 或码属性。

(3) 不包含在任何候选码中的属性称为**非主属性** (Nonprime Attribute) 或非码属性 (Non-key Attribute)。

(4) 在关系模式中, 最简单的情况, 单个属性是码, 称为单码 (Single Key); 最极端的情况, 整个属性集合是码, 称为**全码** (All-Key)。



码



□ 例如：

- 在关系模式 $S(sno, sdept, sage)$ 中, sno 是单码。
- 在关系模式 $SC(sno, cno, score)$ 中, 属性组合 (sno, cno) 是码。
- 在关系模式 “签约(演员名, 制片公司名, 电影名)” 中, 由于一个制片公司可以为一部电影和多个演员签约, 一个演员可以和多个制片公司签约饰演多部电影中的角色, 一部电影可由不同的制片公司制作, 所以此关系模式的码为(演员名, 制片公司名, 电影名), 即为全码。



【定义 3.5】 关系模式 R 中属性或属性组 X 并非 R 的码，但 X 是另一个关系模式的码，则称 X 是 R 的**外部码** (Foreign key)，也称外码。
主码又和外部码一起提供了表示**关系间联系**的手段。

- 如在 $SC(sno, cno, score)$ 中， sno 不是码，但 sno 是关系模式 $S(sno, sdept, sage)$ 的码，则 sno 是关系模式 SC 的外码。



范式以及应用案例

□ **非规范化**：一个关系中存在还可以再分的数据项

- 第一种是关系中具有组合数据项。
- 第二种是关系中具有多值数据项。

□ **规范化**一个关系中的所有分量都是**不可再分**的数据项

- 当关系中不存在组合数据项和多值数据项，只存在不可分的数据项。

□ **范式**

- 利用**规范化理论**，使关系模式的函数依赖集满足特定的要求，满足特定要求的关系模式称为范式。



范式以及应用案例

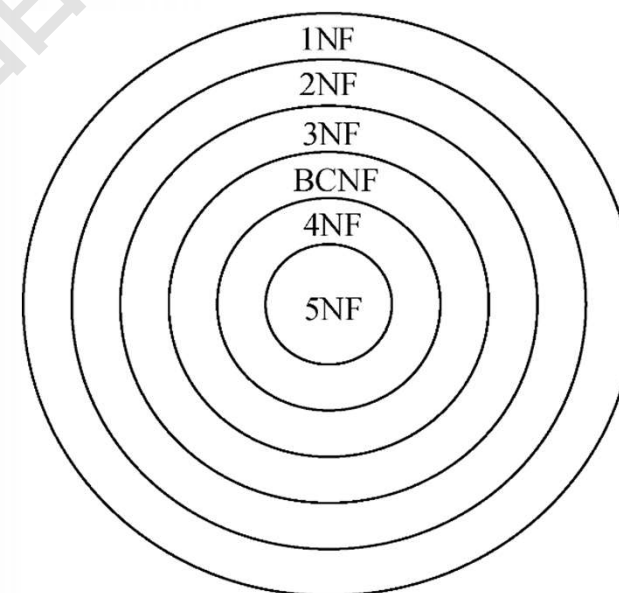
□ 关系按其规范化程度从低到高可分为 **5 级范式**(Normal Form), 分别称为 1NF、2NF、3NF(BCNF)、4NF、5NF。

□ 规范化程度较高者必是较低者的**子集**, 即

$5NF \subseteq 4NF \subseteq BCNF \subseteq 3NF \subseteq 2NF \subseteq 1NF$

□ **规范化:**

- 一个低一级范式的关系模式, 通过**模式分解**可以转换成若干高一级范式的关系模式的集合。





规范化的过程

- 1) 标识表中的所有**候选键**;
- 2) 标识表中的所有**函数依赖**;
- 3) 检查函数依赖的**决定因子**。如果某决定因子不是候选键，则表的结构就有问题。可以通过如下方式实现：
 - (1) 把函数依赖的列放在它自己的新表中;
 - (2) 把函数依赖的决定因子作为新表的主键;
 - (3) 将决定因子的副本作为原表中外键;
 - (4) 在新表和原表之间创建参照完整性约束。
- 4) 根据需要，多次重复步骤3，直至每个表的决定因子都是候选键。



1NF



□ 1NF的定义

如果一个关系模式R的所有属性都是不可分的基本数据项，则 $R \in 1NF$ 。俗称“**表中无表**”。

□ **第一范式是对关系模式的最起码的要求。**不满足第一范式的数据库模式不能称为关系数据库。

□ 但是满足第一范式的关系模式并不一定是一个好的关系模式。



1NF



□ 什么是一个好的模式

- 设关系模式 $R\langle U, F \rangle \in 1NF$ ，如果对于R的每个函数依赖 $X \rightarrow Y$ ，若Y不属于X，则X必含有候选码，那么这个关系模式就是一个好的模式（**BCNF**）。

□ 数据依赖：

函数依赖 1-2-3-BCNF

多值依赖 4NF

连接依赖 5NF



【定义 3.6】 如果关系模式 $R(U, F) \in 1NF$, 且 R 中的每个**非主属性完全函数依赖**于 R 的某个候选码, 则 R 满足第二范式(Second Normal Form), 记作 $R \in 2NF$ 。

□ 例:

- 关系模式 S-L-C (Sno, Sdept, Sloc, Cno, Grade)
Sloc为学生住处, 假设每个系的学生住在同一个地方。
- S-L-C的码为(Sno, Cno)。
- 函数依赖有:

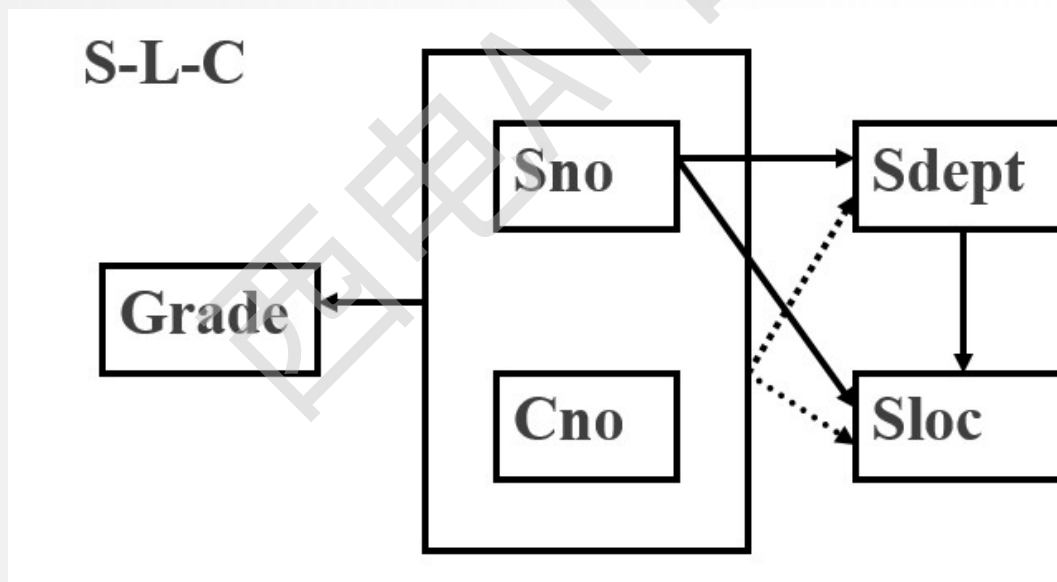
$(\text{Sno}, \text{Cno}) \xrightarrow{f} \text{Grade}$
 $\text{Sno} \rightarrow \text{Sdept}$
 $(\text{Sno}, \text{Cno}) \xrightarrow{P} \text{Sdept}$
 $\text{Sno} \rightarrow \text{Sloc}$
 $(\text{Sno}, \text{Cno}) \xrightarrow{P} \text{Sloc}$
 $\text{Sdept} \rightarrow \text{Sloc}$



2NF



- 非主属性Sdept和Sloc**部分函数依赖**于码(Sno, Cno)
- 关系模式S-L-C不属于2NF





2NF



□ 一个关系模式不属于2NF，会产生以下问题：

➤ **插入异常**

- 如果插入一个新学生，但该生未选课，即该生无Cno，由于插入元组时，必须给定码值，因此插入失败。

➤ **删除异常**

- 如果S4只选了一门课C3，现在他不再选这门课，则删除C3后，整个元组的其他信息也被删除了。

➤ **修改复杂**

- 如果一个学生选了好几门课，则Sdept, Sloc被存储了多次。如果该生转系，则需要修改所有相关的Sdept和Sloc，造成修改的复杂化。



□ 出现这种问题的**原因**

➤ 例子中有两类非主属性：

- 一类如Grade，它对码**完全函数依赖**
- 另一类如Sdept、Sloc，它们对码**不是完全函数依赖**

□ **解决方法：**

➤ 用投影分解把关系模式S-L-C分解成两个关系模式

- SC(Sno,Cno,Grade)
- S-L(Sno,Sdept,Sloc)



3NF



【定义 3.7】 如果关系模式 $R(U, F) \in 2NF$, 且每个**非主属性**都**不传递函数依赖**于任何候选码, 则 R 满足第三范式(Third Normal Form), 记作 $R \in 3NF$ 。

□ 例:

- SC没有传递依赖, 因此 $SC \in 3NF$
- S-L中 $Sno \rightarrow Sdept$ ($Sdept \nrightarrow Sno$), $Sdept \rightarrow Sloc$, 可得 $Sno \xrightarrow{\text{传递}} Sloc$ 。
- 解决的办法是将S-L分解成
- S-D($Sno, Sdept$) $\in 3NF$
- D-L($Sdept, Sloc$) $\in 3NF$



- **BCNF**(Boyce Codd Normal Form)是由 Boyce 和 Codd 提出的, 比上述的 3NF 又进了一步, 通常认为 BCNF 是**修正的第三范式**, 有时也称为**扩充的第三范式**。
- **【定义 3.8】** 关系模式 $R(U, F) \in 1NF$, 若 $X \rightarrow Y$ 且 $Y \subseteq X$ 时, X 必含有码, 则 $R(U, F) \in BCNF$ 。也就是说, 关系模式 $R(U, F)$ 中, **若每个决定因素都包含码**, 则 $R(U, F) \in BCNF$ 。
- 由 BCNF 的定义可以得出结论, 一个满足 BCNF 的关系模式有:
 - (1)所有**非主属性**对每一个码都是**完全函数依赖**。
 - (2)所有的主属性对每一个**不包含它的码**, 也是**完全函数依赖**。
 - (3)没有任何属性完全函数依赖于非码的任何一组属性。



BCNF



□ 例1:

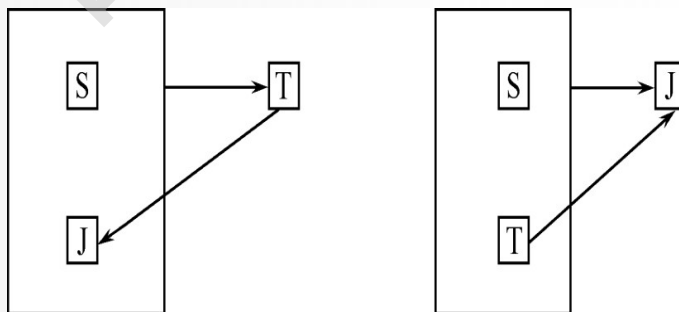
- 设关系模式 $SC(U, F)$, 其中, $U=\{SNO, CNO, SCORE\}$, $F=\{(SNO, CNO) \rightarrow SCORE, (CNO, SCORE) \rightarrow SNO\}$.
- SC 的候选码为 (SNO, CNO) 和 $(CNO, SCORE)$, 决定因素中都包含码, 没有属性对码传递依赖或部分依赖, 所以 $SC \in BCNF$.



BCNF



- **例2**：设关系模式 $STJ(S, T, J)$ ，其中， S ：学生， T ：教师， J ：课程。每位教师只教一门课，每门课有若干教师，某一学生选定某门课，就对应一位固定的教师。
- 由语义可得到如下的函数依赖： $(S, J) \rightarrow T$ ， $(S, T) \rightarrow J$ ， $T \rightarrow J$ 。该关系模式的候选码为 (S, J) ， (S, T) 。
 - 因为**没有任何非主属性对码传递依赖或部分依赖**， $STJ \in 3NF$ 。
 - 因为 T 是决定因素，而 **T 不包含码**，所以 $STJ \notin BCNF$





3.3 数据库设计的规范化

- 关系数据库中，对关系模式的**基本要求**是满足**第一范式**。
- 规范化程度过低的关系不一定能够很好地描述现实世界。
 - 可能存在插入异常、删除异常、修改复杂、数据冗余等问题。
 - **解决方法**就是对其进行规范化，转换成高级范式。



3.3 数据库设计的规范化

□ 规范化的**基本思想**

- 是**逐步消除**数据依赖中不合适的部分，使模式中的各关系模式达到某种程度的“分离”。

□ 即采用“**一事一地**”的模式设计原则

- 让一个关系描述一个概念、一个实体或者实体间的一种联系。
- 若多于一个概念就把它“**分离**”出去。

□ 因此，规范化实质上是概念的单一化。



3.3 数据库设计的规范化

□ 关系模式规范化的基本步骤



□ 规范化的基本原则是：**由低到高，逐步规范，权衡利弊，适可而止。**



第3章 关系模型与关系规范化理论

- 3.1 关系模型及其定义
- 3.2 关系代数及其运算
- 3.3 数据库设计的规范化
- 3.4 知识点小结
- 课堂测试



3.4 知识点小结

本章知识小结:

- 关系模型的数据结构
- 关系的三类完整性约束
- 关系的操作
- 关系代数中传统的集合运算以及专门的关系运算
- 关系规范化理论



第3章 关系模型与关系规范化理论

- 3.1 关系模型及其定义
- 3.2 关系代数及其运算
- 3.3 数据库设计的规范化
- 3.4 知识点小结
- 课堂测试



课堂测试



□ 课堂测试:

见教材第77-78页27与28题,

27. 使用关系代数表达式表示查询;

28. 写出模式的基本函数依赖以及3NF模式集。



Thanks!

See you